Gestión de importación de productos

Informe técnico del proyecto integrador

Davis Rivera
2 de noviembre de 2026

Resumen

Aplicación web para incorporar productos desde archivos CSV, consultar el catálogo y revisar los resultados de cada importación. La solución utiliza PHP con MVC, dos microservicios Spring Boot, MySQL, Eureka y Spring Cloud Gateway.

La aceptación se determina por producto: los registros válidos se almacenan y los rechazados conservan un motivo asociado a su fila. El historial permite consultar los resultados después de finalizar la carga.

Objetivo

Separar la gestión de productos del procesamiento de archivos, manteniendo comunicación HTTP real entre servicios y trazabilidad del resultado presentado al usuario.

Repositorio

github.com/Drgdavis/Proyecto-Programacion-Final

El código fuente, los scripts SQL, los archivos CSV, la documentación de las API y las evidencias se encuentran en el repositorio.

1. Arquitectura y responsabilidades

Componente	Puerto	Responsabilidad
PHP MVC	8000	Interfaz, formularios y presentación de respuestas.
Gateway	8080	Entrada del frontend y rutas explícitas hacia las API.
Catálogo	8081	Productos, categorías, filtros y reglas de registro.
Importaciones	8082	Lectura CSV, coordinación HTTP e historial.
Eureka	8761	Registro y descubrimiento de instancias.
MySQL	3306	Persistencia en catalogo_db e importaciones_db.

Recorrido de una importación

1. El navegador envía el archivo al frontend PHP.

2. PHP transmite el archivo mediante Gateway al servicio de importaciones.

3. Importaciones valida la estructura, identifica las filas y prepara el lote.

4. Un RestClient con balanceador resuelve el nombre catalogo-service mediante Eureka y envía el lote al catálogo.

5. Catálogo evalúa las reglas y devuelve el resultado de cada producto.

6. Importaciones almacena el resumen y los resultados, que PHP presenta en pantalla.

Separación de datos

PHP no accede a MySQL. Importaciones no escribe en las tablas del catálogo. Cada servicio administra su base y sus entidades; el resultado del lote se correlaciona mediante itemId.

Eureka informa las direcciones de las instancias; no transporta el archivo ni reemplaza a Gateway. Las rutas del Gateway son explícitas.

2. Funciones y reglas de datos

Interfaz PHP MVC

Seis pantallas: inicio con indicadores, carga CSV, resultado por fila, catálogo con filtros y paginación, detalle de producto e historial de importaciones. La estructura separa controladores, modelos, vistas y cliente HTTP.

Contrato del archivo

CSV codificado en UTF-8, separado por comas y con los siguientes encabezados en el orden indicado:

```
codigo,nombre,categoria,precio,existencia
DEMO001,"Teclado, mecanico",Perifericos,350.00,20
```

Dato	Regla
Código	Obligatorio, único en catálogo.
Nombre	Obligatorio.
Categoría	Obligatoria y existente en catálogo.
Precio	Mayor que cero; hasta 10 enteros y 2 decimales.
Existencia	Entero entre 0 y 2147483647.
Archivo	Hasta 5 MB; solicitud hasta 6 MB.

El lector admite comas y comillas escapadas en campos entrecomillados. La fila informada corresponde al registro lógico CSV; el primer producto es la fila 2.

Persistencia y aceptación parcial

catalogo_db contiene categorías y productos. importaciones_db contiene importaciones y resultados por fila. El catálogo aplica una transacción independiente por producto, lo que permite conservar aceptados aunque otro producto sea rechazado. Las restricciones UNIQUE, FOREIGN KEY y CHECK complementan las validaciones de aplicación.

Estados del proceso

PROCESANDO: intento en curso. **COMPLETADA:** todos los registros aceptados. **COMPLETADA_CON_ERRORES:** evaluación finalizada con rechazos. **FALLIDA:** archivo no procesable o fallo técnico; puede existir un resultado no confirmado.

3. API, tecnologías y ejecución

Servicio	Operaciones principales
Catálogo	GET /api/categorias; GET /api/productos; GET /api/productos/{id}; POST /api/productos/lote.
Importaciones	GET y POST /api/importaciones; GET /api/importaciones/{id}; GET /api/importaciones/{id}/errores.

Los filtros del catálogo admiten código, nombre y categoría. La API pagina desde 0 y acepta tamaños de 1 a 100. Los errores utilizan estado HTTP y mensajes comprensibles. Swagger documenta ambas API; Logback conserva método, ruta, estado y duración de las solicitudes.

Entorno

Java 21; Spring Boot 4.0.9; Spring Cloud 2025.1.4; Springdoc 3.0.3; Apache Commons CSV 1.14.1. Entorno probado: PHP 8.5.12 con cURL y MySQL 8.4.13. Maven se ejecuta mediante los wrappers incluidos. H2 se utiliza únicamente en pruebas.

Configuración y arranque

1. Crear los esquemas con database/01_catalogo.sql y database/02_importaciones.sql.
2. Configurar los usuarios mediante las plantillas SQL con claves locales.
3. Definir CATALOGO_DB_PASSWORD e IMPORTACIONES_DB_PASSWORD o introducirlas cuando el script las solicite.
4. Iniciar los cinco procesos en terminales separadas, con Eureka primero.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass
-File .\scripts\Iniciar-Servicio.ps1 -Servicio Eureka
```

El comando anterior se introduce en una sola línea. Repetir en otras terminales con Catalogo, Importaciones, Gateway y Frontend como valor de Servicio. Las terminales deben permanecer abiertas.

Interfaz: http://localhost:8000/
Eureka: http://localhost:8761/
Swagger: http://localhost:8081/swagger-ui.html y http://localhost:8082/swagger-ui.html

4. Verificación y alcance

Verificación	Resultado registrado
Maven: catálogo	5 pruebas; sin fallos ni errores.
Maven: importaciones	22 pruebas; sin fallos ni errores.
Maven: Gateway	4 pruebas; sin fallos ni errores.
Maven: Eureka	1 prueba; sin fallos ni errores.
Total Java	32 pruebas; cuatro módulos con BUILD SUCCESS.
Integración HTTP	33 comprobaciones aprobadas en la ejecución registrada.
Frontend	30 comprobaciones de pantallas, parámetros, métodos y plantilla.
Sintaxis y registro	PHP sin errores sintácticos; registro y disponibilidad de servicios comprobados.

Los resultados Java corresponden a los reportes Surefire del 2 de noviembre de 2026. Las 33 comprobaciones HTTP corresponden a la ejecución de integración registrada previamente; no son métodos JUnit adicionales. Las capturas de la siguiente página documentan la aplicación local del 31 de octubre.

Comandos reproducibles

```
.\scripts\Verificar-Proyecto.ps1
.\pruebas\Verificar-Integracion.ps1
.\frontend-php\pruebas\Verificar-Frontend.ps1
.\eureka-server\pruebas\Verificar-Eureka.ps1
```

Estos scripts se ejecutan con PowerShell desde la raíz del repositorio. El script de integración crea dos productos e intentos de importación con códigos nuevos. Las pruebas del frontend no cargan archivos ni crean datos de negocio.

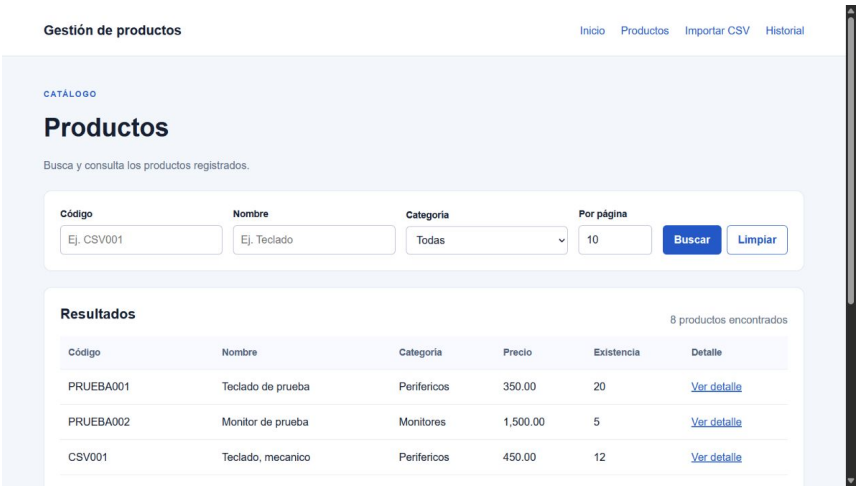
Limitaciones técnicas

La solución se verifica como demostración local. No se ha comprobado su instalación en otra computadora. La carga es síncrona y en memoria. No hay transacción distribuida, recuperación automática de intentos interrumpidos ni reintento automático de resultados inciertos: una respuesta perdida puede corresponder a productos ya guardados. El historial conserva fila, código y motivo, no el archivo completo.

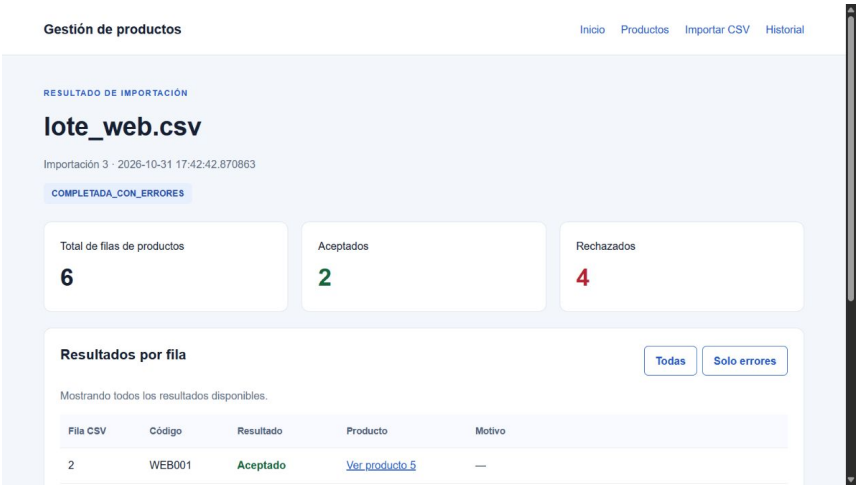
La ausencia de una ruta Gateway no bloquea el acceso directo a un puerto. No se implementó autenticación. Las credenciales locales deben permanecer fuera del repositorio y las claves expuestas requieren rotación.

5. Evidencias y documentación

Capturas históricas de la ejecución local. Los contadores reflejan los datos existentes en ese momento y pueden variar con las pruebas.



Catálogo: filtros, tabla y acceso al detalle.



Resultado: resumen de carga y aceptación o rechazo por fila.

Referencias del proyecto

Enunciado: Proyecto integrador de Programación IV, Universidad Mesoamericana, entrega del 2 de noviembre de 2026.
README.md: instalación y ejecución.
docs/ARQUITECTURA.md y docs/API.md: diseño y contratos.
docs/MANUAL_USUARIO.md: pantallas y operación del usuario.
docs/PRUEBAS_INTEGRACION.md y docs/evidencias/: resultados y evidencias.
docs/LIMITACIONES.md: alcance técnico.